

Today 12:04 PM

</> 05d36618-2e78-484f-acdf-afe5bb434...
Python

I am working on a college assignment called "Defender Repair Lab".

This is a single-file Pygame Defender-lite game. I must use an LLM as my debugging and pair-programming partner.

The assignment has 4 tasks:

1. Fix the radar bug: the radar must show the player, humanoids, and landers using their absolute world positions across the entire wrapping world.
2. Implement `sky_color(wave)`.

Show more ▾

I've read the entire `game.py`. The Task 2 function is currently empty:

</> Python



Run

```
def sky_color(wave):
    """Return an (r, g, b) sky colour for the current wave, or None for the default."""
    pass
```

I'll modify **only this function**, leaving your completed radar fix and all other game logic unchanged.

05d36618-2e78-484f-acdf-afe5bb4...



Now implement Task 2 from the Defender Repair Lab README.

Implement `sky_color(wave)`.

Please inspect the existing `game.py` and modify only the empty `sky_color(wave)` function.

It should return an (r, g, b) color or None, and the sky should gradually brighten or shift hue as the wave number increases.

Do not modify unrelated game logic.

Show more ▾

Worked for 12s >

Task 2 is implemented, and the Python file passes syntax validation.

Exactly what I changed

I changed **only** the body of `sky_color(wave)` :

`</>` Python



▶ Run

```
def sky_color(wave):  
    """Return an (r, g, b) sky colour for the current wave, or None for the default."""
```

```
brightness = min(80, max(0, wave - 1) * 8)
return (5 + brightness // 2, 5 + brightness // 3, 20 + brightness)
```

The function now:

- Starts with a dark blue sky at **Wave 1**: (5, 5, 20)
- Gradually increases brightness as the wave increases.
- Uses `min(80, ...)` so the brightness increase is capped and the RGB values stay reasonable.
- Always returns a valid (r, g, b) tuple.
- Does **not** affect the radar, player, humanoids, landers, scoring, or other game logic.

The existing `draw()` already uses `sky_color(self.wave)`, so this function automatically changes the background as waves progress.  05d36618-2e78-484f-acdf-afe5bb4...

Modified file

 [Download the completed game.py](#)

You can now run this file and test **Task 2**.

 **05d36618-2e78-484f-acdf-afe5bb434bf4.py**
Code

Sources



Task 2 (sky_color) is implemented and tested successfully.

Now implement Task 3 from the Defender Repair Lab README.

Implement `on_humanoid_rescued(humanoid)`.

Please inspect the current `game.py` carefully before making changes.

The existing `update()` function already:

Show more ▾

Worked for 31s >

↑ Share ...

Task 3 is implemented.

What I changed

The existing rescue logic already gives the player **500 points**:

Python



Run

```
self.score += 500
on_humanoid_rescued(humanoid)
```

So I did **not** add another score increase.

I made `on_humanoid_rescued(humanoid)` start a short visual celebration:

Python



Run

```
def on_humanoid_rescued(humanoid):
    """Called when the player catches a falling humanoid; add a bonus or celebration here."""
    humanoid.rescue_flash = 0.6
```

This gives the rescued humanoid a **0.6-second** `rescue_flash`.

I added the minimal supporting logic to:

1. Decrease the flash timer each frame.
2. Display `+500` above the rescued humanoid while the timer is active.

So when you rescue a humanoid, the existing **500 points** are awarded and a visible **+500** appears briefly above the humanoid.

What I did NOT change

- Radar fix — unchanged
- `sky_color()` — unchanged
- Existing rescue detection — unchanged
- Existing 500-point score — unchanged
- Lander logic — unchanged
- Player logic — unchanged
- Bonus-life logic — unchanged

The file also passes a Python syntax check.

[`</>` Download the updated game.py](#)

`</>` 05d36618-2e78-484f-acdf-afe5bb434bf4.py
Code



Task 3 (on_humanoid_rescued) is implemented and tested successfully.

Now implement Task 4 from the Defender Repair Lab README.

Implement `bonus_life_threshold()`.

Please inspect the current `game.py` carefully before making changes.

The existing `update()` function already handles the bonus-

Show more ▾

Exactly what I changed

I changed **only** the empty `bonus_life_threshold()` function:

</> Python



▶ Run

```
def bonus_life_threshold():  
    """Return a score value at which the player earns an extra life, or None to disable bonus  
    return 10000
```

So the function now returns the README-specified **10,000-point threshold**.

Why this works

The existing `update()` logic already handles everything else:

</> Python



▶ Run

```
threshold = bonus_life_threshold()  
  
if threshold and self.score // threshold > self.bonus_awarded:  
    self.bonus_awarded = self.score // threshold  
    self.lives += 1
```

Therefore:

- At **10,000 points** → +1 life
- At **20,000 points** → +1 additional life
- At **30,000 points** → +1 additional life
- `self.bonus_awarded` prevents repeatedly awarding the same bonus life.

I did **not** modify:

- Radar fix
- `sky_color()`
- Rescue celebration
- Existing scoring
- Existing bonus-life bookkeeping
- Any unrelated game logic

I also verified that the updated Python file passes a syntax check. Python functions return the value specified by their `return` statement, which is exactly what this implementation

uses.  Python docume...

</> **Download the completed game.py** ↕